UNIVERSITY OF NEGROS OCCIDENTAL – RECOLETOS, INC.
COLLEGE OF INFORMATION TECHNOLOGY
#51 Lizares St, Bacolod City, 6100 Negros Occidental, Philippines

Predicting Next-Year Fault Occurrence in Global Undersea Cable Systems Using Infrastructure and Operational Indicators

Code and Results Documentation

Introduction to Data Science

Submitted to:

Mr. Maxil S. Urocay, MSCS candidate

And to the Faculty of the

College of Information Technology

University of Negros Occidental - Recoletos, Inc.

In Partial Fulfillment

Of the Requirements for the Course

COPE0683X

Submitted By

Group 5 - K44

Bayoguing, Jose Vincent III

Cruz, Aurum Fuego

Sapio, John Rey

October 31, 2026

Component 1 Data Loading and Quality Assessment

- The loader preserves the valid category "None". The saved audit reports the dimensions, labels, and duplicate checks. The supplied panel is procedurally generated synthetic data; these are not authenticated observations of real cable systems.

Source excerpt src/data_loading.py

```
def load_csv(path: Path, *, target_nullable: bool = False) -> pd.DataFrame:
    """Read CSV while preserving literal ``None`` as a category."""
    frame = pd.read_csv(path, keep_default_na=False, na_values=[])
    for col in frame.columns:
        if col in NUMERIC_COLUMNS:
            if target_nullable and col == config.TARGET_COLUMN:
                frame[col] = pd.to_numeric(frame[col].replace({"": pd.NA}),
    errors="coerce")
            else:
                frame[col] = pd.to_numeric(frame[col], errors="coerce")
    return frame
```

Saved output outputs/tables/raw_data_quality_summary.csv

```
check                      value
-------------------------  -----
rows                       4821
columns                    21
unique_cables              500
min_year                   2015
max_year                   2026
labeled_rows               4321
positive_labels            820
negative_labels            3501
positive_prevalence        0.190
duplicate_rows             0
duplicate_cable_year       0
chokepoint_literal_None    3986
design_life_unique         1
low_redundancy_equivalence True
temporally_valid_labeled   4315
```

Component 2 Temporal Label Checks and Cleaning

- A next-year label is eligible only when the next observed year is consecutive. Records remain in the cleaned panel even when they cannot enter supervised modeling.

Source excerpt src/data_validation.py

```
def add_temporal_label_audit(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    ordered = out.sort_values([config.ID_COLUMN, config.YEAR_COLUMN])
    out["next_observed_year"] =
    ordered.groupby(config.ID_COLUMN)[config.YEAR_COLUMN].shift(-1).reindex(out.index)
    out["has_consecutive_next_year"] = (out["next_observed_year"] ==
    out[config.YEAR_COLUMN] + 1)
    out["label_temporally_valid"] = out[config.TARGET_COLUMN].notna() &
    out["has_consecutive_next_year"]
    return out
```

Source excerpt scripts/run_pipeline.py

```
def clean_data(audited: pd.DataFrame) -> pd.DataFrame:
    out = audited.copy()
    out["chokepoint"] = out["chokepoint"].astype(str).replace({"": "None"})
    out["exclusion_reason"] = ""
    out.loc[out[config.TARGET_COLUMN].isna(), "exclusion_reason"] = "unlabeled final
    observation"
    out.loc[out[config.TARGET_COLUMN].notna() & ~out["has_consecutive_next_year"],
    "exclusion_reason"] = "unresolved non-consecutive next observation"
    return out
```

Saved cleaning summary selected fields

```
rows_before: 4821
rows_after: 4821
supervised_eligible_rows: 4315
literal_None_preserved: 3986

Six non-consecutive labels excluded from supervised data.
500 final observations have no forward label.
```

Component 3 Exploratory Analysis and Statistical Summaries

- Descriptive outputs cover observations through 2024, before the reserved final-test year. Numeric summaries and category counts describe the development-period panel, including rows not eligible for supervised analysis.

Saved output eda/descriptive_statistics.csv selected variables

Variable	N	Mean	Median	Min	Max
age_years	3860	11.04	11	0	26
length_km	3860	6823.73	5761	207	19976
design_capacity_tbps	3860	289.29	182.90	16	2166.90
lit_capacity_tbps	3860	184.29	125.65	16	1110
utilization_pct	3860	61.54	61.10	1	100
route_redundancy	3860	45.44	44	27	75

Saved output eda/ocean_route_summary.csv

ocean_route	count	proportion
Intra-Asia	648	0.168
Intra-Europe	475	0.123
Trans-Atlantic	453	0.117
Africa-Coastal	423	0.110
Trans-Pacific	416	0.108
EMEA-Asia	383	0.099
Intra-Pacific-Islands	368	0.095
Indian-Ocean	362	0.094
Americas	332	0.086

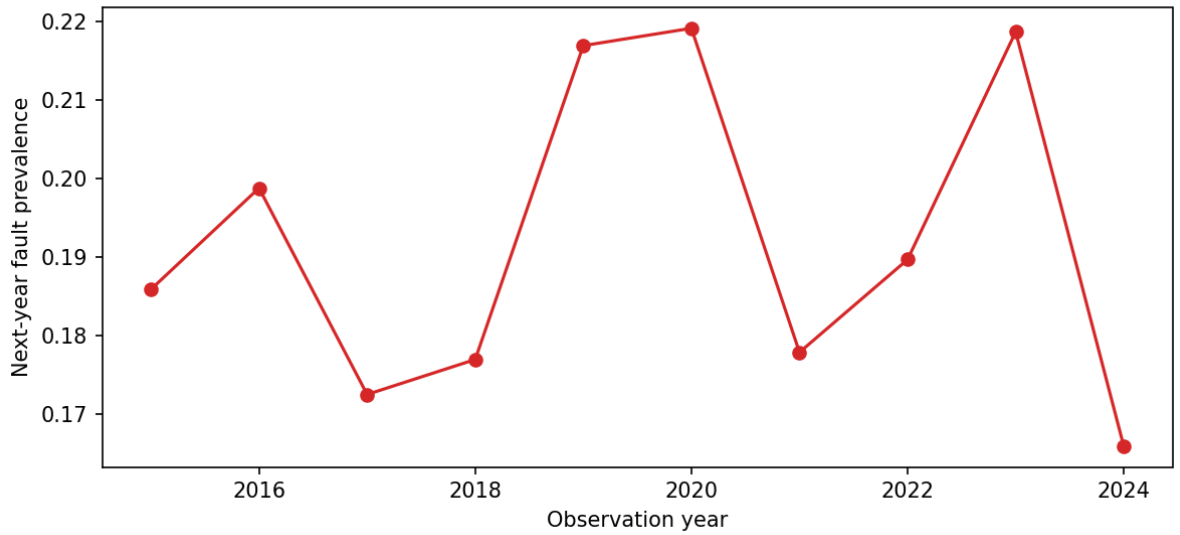
Saved output eda/operator_type_summary.csv

operator_type	count	proportion
consortium	1578	0.409
carrier	1093	0.283
hyperscaler	836	0.217
government	353	0.091

Component 4 Exploratory Analysis Graphical Outputs

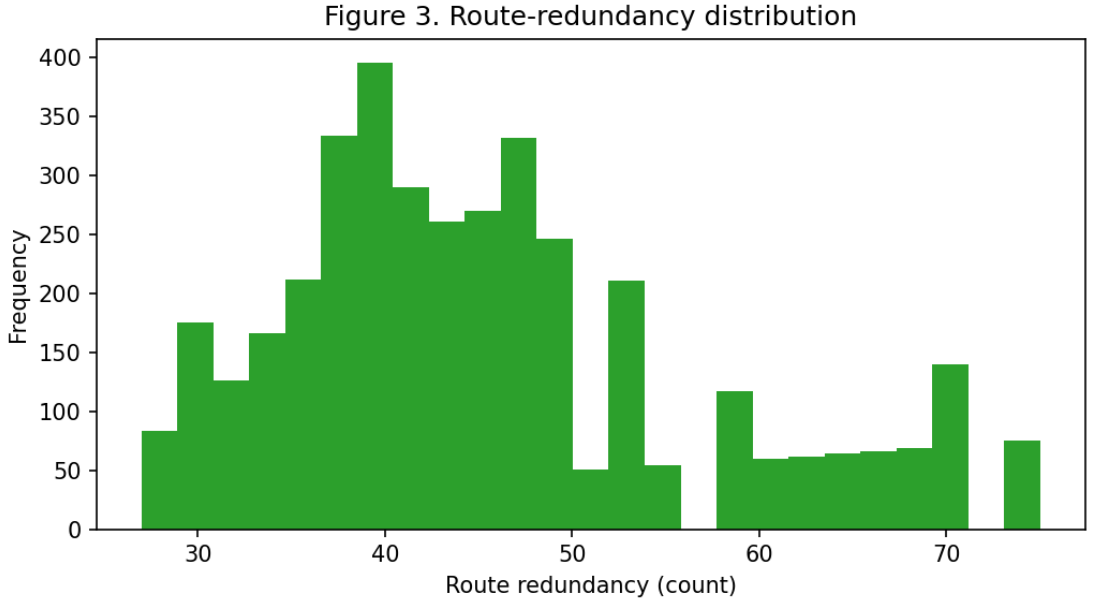
- The saved figures show annual next-year fault prevalence and route redundancy in the development-period panel. They are descriptive views of the supplied dataset, not observed global cable fault rates.

Figure 2. Annual next-year fault prevalence



Saved figure outputs/figures/eda/figure_02_annual_prevalence.png

Figure 3. Route-redundancy distribution



Saved figure outputs/figures/eda/figure_03_route_redundancy.png

Component 5 Statistical Association Analysis

- Saved tests compare eligible development records with and without next-year faults. Associations are exploratory and non-causal; repeated cable observations and multiple comparisons limit ordinary p-value interpretation.

Source excerpt src/statistics.py

```
def numeric_comparisons(df):
    rows=[]
    for col in config.NUMERICAL_PREDICTORS:
        a=df.loc[df[config.TARGET_COLUMN]==0,col].dropna();
    b=df.loc[df[config.TARGET_COLUMN]==1,col].dropna()
        stat,p=mannwhitneyu(a,b,alternative="two-sided")
        rows.append({"variable":col,"test":"Mann-Whitney U","statistic":stat,"p_value":p,
    "median_no_fault":a.median(),"median_fault":b.median(),"n":len(a)+len(b)})
    return pd.DataFrame(rows)
```

Saved numeric comparisons selected fields n = 3851

Variable	Median 0	Median 1	p
age_years	10	13	3.12e-08
fiber_pairs	8	8	3.74e-01
length_km	5630	6810.500	3.84e-11
n_landing_countries	5	5	1.29e-01
design_capacity_tbps	196.600	168	5.13e-04
lit_capacity_tbps	128.100	115.450	1.10e-01
utilization_pct	60.200	64.950	1.70e-05
protected_burial	1	0	9.19e-09
route_redundancy	44	41	8.39e-06
fault_this_year	0	0	6.06e-20

Saved categorical associations selected fields

Variable	Cramers V	p	N
ocean_route	0.155	1.09e-16	3851
operator_type	0.026	4.49e-01	3851
chokepoint	0.146	6.03e-15	3851

Component 6 Predictor Preparation and Chronological Splits

- The primary analysis uses 13 approved predictors. Imputation and encoding belong inside each fitted pipeline. The final-test partition is reserved and is not scored here.

Source excerpt src/config.py

```
CATEGORICAL_PREDICTORS = ["ocean_route", "operator_type", "chokepoint"]
NUMERICAL_PREDICTORS = [
    "age_years", "fiber_pairs", "length_km", "n_landing_countries",
    "design_capacity_tbps", "lit_capacity_tbps", "utilization_pct",
    "protected_burial", "route_redundancy", "fault_this_year",
]
PREDICTORS = CATEGORICAL_PREDICTORS + NUMERICAL_PREDICTORS
TRAIN_YEARS = list(range(2015, 2023))
VALIDATION_YEARS = [2023, 2024]
FINAL_TEST_YEAR = 2025
```

Source excerpt src/preprocessing.py

```
def chronological_split(df: pd.DataFrame) -> Dict[str, pd.DataFrame]:
    return {
        "train": df[df[config.YEAR_COLUMN].isin(config.TRAIN_YEARS)].copy(),
        "validation": df[df[config.YEAR_COLUMN].isin(config.VALIDATION_YEARS)].copy(),
        "final_test": df[df[config.YEAR_COLUMN] == config.FINAL_TEST_YEAR].copy(),
    }
```

Saved split summary label counts omitted for the locked final test

```
Partition   First year  Last year  Rows
----------  ----------  ---------  ----
train       2015        2022       2960
validation  2023        2024       891
final_test  2025        2025       464
```

Component 7 Model Development Code

- The repository defines a majority-class baseline, logistic regression, decision tree, and random forest. These excerpts show the saved candidate specifications and the training-versus-validation workflow; the documentation build does not retrain them.

Source excerpt src/modeling.py

```
def candidate_estimators() -> Dict[str, object]:
    return {
        "dummy_baseline": DummyClassifier(strategy="prior"),
        "logistic_regression": LogisticRegression(C=1.0, class_weight="balanced",
    max_iter=2000, random_state=config.RANDOM_SEED),
        "decision_tree": DecisionTreeClassifier(max_depth=6, min_samples_leaf=10,
    class_weight="balanced", random_state=config.RANDOM_SEED),
        "random_forest": RandomForestClassifier(n_estimators=250, max_depth=10,
    min_samples_leaf=3, max_features="sqrt", class_weight="balanced",
    random_state=config.RANDOM_SEED, n_jobs=-1),
    }
```

Source excerpt src/modeling.py

```
def fit_and_compare(train: pd.DataFrame, validation: pd.DataFrame, *, predictors=None,
    artifact_suffix=""):
    predictors = list(predictors or config.PREDICTORS)
    X_train, y_train = train[predictors], train[config.TARGET_COLUMN].astype(int)
    X_val, y_val = validation[predictors], validation[config.TARGET_COLUMN].astype(int)
    rows = []; fitted = {}
    for name in candidate_estimators():
        pipe = make_pipeline(name, predictors=predictors); pipe.fit(X_train, y_train)
        metrics = evaluate_pipeline(pipe, X_val, y_val)
        metrics.update({"model": name, "parameters":
    json.dumps(pipe.named_steps["model"].get_params(), default=str, sort_keys=True)})
        rows.append(metrics); fitted[name] = pipe
        joblib.dump(pipe, config.MODELS_DIR / f"{name}{artifact_suffix}.joblib")
    return pd.DataFrame(rows), fitted
```

Component 8 Validation Performance and Model Selection

- Saved scores evaluate 891 records from 2023-2024, including 170 positive labels. The primary model selection uses average precision. The majority baseline has high accuracy because it predicts no faults, but it detects none of the positive records.

Source excerpt src/modeling.py

```
def select_model(comparison: pd.DataFrame) -> str:
    ranked = comparison.sort_values(["pr_auc", "f1", "recall"], ascending=False,
    kind="mergesort")
    return str(ranked.iloc[0]["model"])
```

Saved validation metrics fault class is the positive class

```
Model               Prec   Recall  F1     AP     ROC AUC  Acc
------------------  -----  ------  -----  -----  -------  -----
dummy_baseline      0      0       0      0.191  0.500    0.809
logistic_regression 0.253  0.518   0.340  0.311  0.625    0.616
decision_tree       0.243  0.541   0.335  0.239  0.580    0.590
random_forest       0.299  0.406   0.344  0.317  0.636    0.705
```

Saved model selection outputs/metrics/model_selection.json

```
selected_model: random_forest
development_partitions: train 2015-2022; validation 2023-2024
limitations: Results describe the supplied synthetic dataset and are not real-world cable
     evidence.
```

AP denotes average precision, stored as pr_auc in the project CSV. These are validation results, not final-test performance estimates.

Saved expanding-window cross-validation training years only

```
Model               Mean AP  SD AP  Folds
------------------  -------  -----  -----
decision_tree       0.279    0.007  4
dummy_baseline      0.201    0.020  4
logistic_regression 0.341    0.027  4
random_forest       0.363    0.021  4
```

Component 9 Infrastructure and Operational Indicators

- The supplementary comparison separates nine infrastructure predictors and four operational predictors, then compares them with the combined set. It reuses the development validation period and does not replace the frozen primary model selection.

Saved indicator registry selected columns

feature	indicator_group
ocean_route	infrastructure
chokepoint	infrastructure
age_years	infrastructure
fiber_pairs	infrastructure
length_km	infrastructure
n_landing_countries	infrastructure
design_capacity_tbps	infrastructure
protected_burial	infrastructure
route_redundancy	infrastructure
operator_type	operational
lit_capacity_tbps	operational
utilization_pct	operational
fault_this_year	operational

Saved indicator-block validation comparison

Group	Model	Inputs	AP	Recall	F1
infrastructure	logistic_regression	9	0.292	0.547	0.353
infrastructure	random_forest	9	0.327	0.382	0.327
operational	logistic_regression	4	0.258	0.441	0.314
operational	random_forest	4	0.257	0.365	0.283
combined	logistic_regression	13	0.311	0.518	0.340
combined	random_forest	13	0.317	0.406	0.344

Infrastructure-only random forest has higher AP in this supplementary comparison (0.327 versus 0.317 combined), while combined random forest has higher recall (0.406 versus 0.382). Repeated validation is not independent confirmation.

Component 10 Model Interpretation Outputs

- The saved random-forest importances rank inputs used by the model. Higher importance does not establish a causal effect or show that an indicator can be changed to prevent faults.

Source excerpt scripts/run_pipeline.py

```
for model_name in ["decision_tree", "random_forest"]:
    est = fitted[model_name].named_steps["model"]
    names = fitted[model_name].named_steps["preprocess"].get_feature_names_out()
    write_csv(pd.DataFrame({"feature": names, "importance":
    est.feature_importances_}).sort_values("importance", ascending=False),
    interp_dir/f"{model_name}_feature_importance.csv")
```

Saved output random_forest_feature_importance.csv first 15 rows

```
feature                              importance
-----------------------------------  ----------
numeric__length_km                   0.13168
numeric__utilization_pct             0.11693
numeric__lit_capacity_tbps           0.09593
numeric__age_years                   0.09243
numeric__route_redundancy            0.08243
numeric__design_capacity_tbps        0.07719
numeric__n_landing_countries         0.06111
numeric__fiber_pairs                 0.05871
numeric__fault_this_year             0.05286
categorical__chokepoint_None         0.03895
numeric__protected_burial            0.03488
categorical__operator_type_consortium  0.01528
categorical__operator_type_carrier   0.01391
categorical__ocean_route_Intra-Europe  0.01247
categorical__ocean_route_Americas    0.01212
```

These are impurity-based importances for encoded model inputs. Correlated variables can share importance, and one-hot categories appear as separate inputs.

Component 11 Validation Error Counts and Final Test Status

- The selected random forest detects 69 of the 170 positive validation records and misses 101. The error counts are recorded development results; no final-test predictions or scores are generated for this document.

Source excerpt src/evaluation.py

```
def evaluate_pipeline(model, X, y) -> Dict[str, Any]:
    pred = model.predict(X)
    prob = model.predict_proba(X)[:, 1] if hasattr(model, "predict_proba") else None
    return classification_metrics(y, pred, prob)

def final_test_guard(allow: bool = False) -> None:
    if not allow:
        raise PermissionError("Final test evaluation is locked. Enable only for
    authorized Final-Term evaluation after model selection is frozen.")
```

Saved validation confusion counts all candidate models

```
model               TN   FP   FN   TP
------------------  ---  ---  ---  --
dummy_baseline      721  0    170  0
logistic_regression 461  260  82   88
decision_tree       434  287  78   92
random_forest       559  162  101  69
```

Selected random forest validation confusion matrix

```
                        Predicted 0   Predicted 1
Actual 0  no next-year fault     559           162
Actual 1  next-year fault        101            69

TN: correct non-fault   FP: false alarm
FN: missed fault       TP: detected fault
```

Saved status outputs/metrics/final_test_status.txt

```
Final test evaluation is locked. Enable only for authorized Final-Term evaluation after
    model selection is frozen.
```

The 2025 predictor-year holdout remains reserved for authorized final evaluation. External real-world context and incident records are not joined to the synthetic CAB identifiers.